

Gravity: A Physics-Inspired $O(N)$ Framework with $O(1)$ Streaming State Across Dimensions

Chia-Wei Chiang
Independent Researcher

May 2026

Abstract

We introduce **Gravity**, a neural network architecture that replaces quadratic self-attention with physics-derived field equations on lattice structures of any dimensionality. Each input element is mapped to K field parameters, compressed to a scalar density via a learnable nonlinear bottleneck, and propagated through multi-scale exponential kernel recurrence in $O(N)$ time. Attention scores are computed only within a fixed-size local window, eliminating any $N \times N$ matrix. The same core mechanism—density bottleneck, multi-scale field propagation, local window interaction, and physics feature extraction—extends from 1D sequences to 2D grids, 3D volumes, and spatio-temporal video tensors by varying only the number of directional parallel scans. For video, propagation operates on the 3D spatio-temporal lattice $T \times H \times W$.

This design trades content-addressable retrieval for bounded-state inference. We characterize the tradeoff through data scaling from 100M to 1B tokens: the cross-entropy loss gap between Gravity and Transformer grows from 3.9% to 9.6% but decelerates sharply (+4.6 pp then +1.1 pp), consistent with convergence toward a stabilizing cost near $\sim 10\%$.

Across eleven experiments in four data regimes (Table 1), Gravity outperforms the strongest tested attention/Transformer-style baseline on three discriminative tasks (sensor, image, volume) and is comparable on two (language at $2.1\times$ fewer parameters; video within 0.8 pp). At seq=65,536, Transformer exceeds A100 memory while Gravity uses 411 MB; streaming 1M tokens, Gravity maintains constant memory (CV=0.14%). Inference state is $\sim 200\times$ smaller than Mamba (fp32) at the cost of $\sim 10\%$ cross-entropy. The $O(1)$ claim refers to streaming inference along causal dimensions; bidirectional spatial processing requires $O(N)$ activation memory. Large-scale results (100M–1B tokens) are single-epoch probes.

1 Introduction

The self-attention mechanism in Transformer architectures computes pairwise interactions between all N tokens, requiring $O(N^2)$ time and memory. This quadratic cost is the primary bottleneck for long-context deployment: at 128K context, a Transformer with $d_{\text{model}} = 4096$ stores $128\text{K} \times 4096 \times 2$ key/value values per layer, corresponding to approximately 4.3 GB per layer in fp32 (2.15 GB in fp16); across many layers the KV-cache alone can reach tens to hundreds of gigabytes. Engineering solutions such as FlashAttention [5] reduce memory overhead but do not reduce the fundamental $O(N^2)$ computation. Sparse attention methods (Longformer, BigBird) sacrifice expressiveness. Linear attention and state-space models (S4, Mamba) achieve $O(N)$ scaling but compress all context into a fixed-size hidden state, resulting in information loss for tasks requiring precise retrieval of distant tokens.

For multi-dimensional data—images, volumetric medical scans, video—the quadratic scaling is even more prohibitive: a 512×512 image yields over 260K positions, a CT volume may have millions of voxels. State-space models such as Mamba are originally formulated for one-dimensional sequences; recent extensions

(VMamba [14]; Mamba-ND [13]) adapt Mamba to 2D/3D by applying directional scans along multiple axes, achieving competitive results on vision benchmarks. However, these approaches propagate high-dimensional state vectors and flatten spatial structure to 1D sequences within each scan layer. CNNs excel at 2D/3D through built-in translation invariance, and 1D convolutions (TCN, WaveNet, ConvS2S) can model sequences, but standard convolutional architectures do not provide the same unified bounded-state field propagation mechanism across language, images, volumes, and video. To our knowledge, no prior architecture compresses representations to scalar density before multi-directional propagation, nor extracts physics-derived features (potential, gradient, curvature) from the propagated field.

We propose an approach rooted in classical field theory that differs substantially from both attention and existing SSM methods. In gravitational physics, point masses do not interact directly—each mass generates a field that propagates through space, and other masses respond to the local field. This indirect, field-mediated interaction achieves global information propagation without pairwise computation. Gravity applies this principle: elements are compressed to a scalar density, propagated via multi-scale causal recurrence, and attended within a fixed local window. The core innovation is the **density bottleneck**: K -dimensional field parameters are compressed to a 1-dimensional (or C -channel) scalar density via a learnable nonlinear projection $\rho = \text{softplus}(W) \cdot \text{params}^2$ before propagation. This scalar compression has a precise theoretical origin: in the lattice gauge theory motivation that led to Gravity, each element’s local degrees of freedom are represented by Lie algebra coefficients a_k associated with the 15 generators T_k of $SU(4)$. The scalar density used in the architecture corresponds to a gauge-invariant generator-energy proxy $\rho \propto \text{Tr}(A^\dagger A)/G$ with $A = \sum_k a_k T_k$, where G denotes the gauge/group dimension ($G = 4$ for $SU(4)$; independent of sequence length N). This reduces to a weighted quadratic form in the coefficients—implemented in this paper as $\rho = \sum_k w_k a_k^2$. The $K = 15$ field parameters in the default configuration correspond to the 15 generators of $SU(4)$. The present work uses this scalar-compressed form; matrix-valued field propagation is discussed as a future direction.

The key insight enabling multi-dimensional operation is that the field propagation—implemented via directional scan decomposition—is inherently dimension-agnostic. In 1D it reduces to a single causal scan; in 2D it decomposes into 4 directional scans ($\pm x, \pm y$); in 3D into 6 directional scans ($\pm x, \pm y, \pm z$). The density bottleneck, local window, and physics features are identical across dimensions—only the number of scan directions changes.

This compression comes at a cost. Gravity trades content-addressable retrieval—the ability to precisely recall specific past tokens—for bounded-state inference. We characterize this tradeoff precisely: the cross-entropy loss gap between Gravity and Transformer grows from 3.9% to 9.6% as training data scales from 100M to 1B tokens, but the rate of increase decelerates sharply (+4.6 pp then +1.1 pp), suggesting convergence to an information cost consistent with stabilization of $\sim 10\%$ rather than unbounded degradation. We interpret this as evidence consistent with a bounded-state information cost rather than an unbounded degradation trend, though fully converged multi-epoch scaling at this parameter scale remains future work.

Contributions.

1. A unified data processing paradigm achieving $O(N)$ computation, $O(N)$ memory, and $O(1)$ autoregressive inference state per layer across 1D sequences, 2D grids, 3D volumes, and spatio-temporal video tensors.
2. A parallel associative scan implementation achieving $13\times$ speedup over sequential scanning on data-center GPUs.
3. Validation across eleven experiments in four data regimes—five discriminative (language, sensor, image,

volume, video), two generative (2D and 3D diffusion), and four efficiency benchmarks—outperforming the strongest tested attention/Transformer-style baseline on three discriminative tasks (sensor, image, volume) and achieving comparable accuracy on the other two (language modeling at $2.1\times$ fewer parameters; video within 0.8 pp); 3D diffusion generates coherent volumes (occupancy/connectivity matching ground truth) and demonstrates DiT OOM at 32K patches while Gravity runs at 0.73 GB.

4. At $\sim 100\text{M}$ -parameter scale (128M-parameter OpenWebText probes), honest characterization of the density bottleneck’s scaling boundary via data scaling from 100M to 1B tokens, revealing that the cross-entropy loss gap is consistent with a stabilizing trend near $\sim 10\%$ rather than growing without bound.
5. Empirical demonstration that Transformer exceeds A100 40GB memory at seq=65,536 while Gravity completes the forward pass using 411 MB, and that Gravity maintains constant memory (CV=0.14%) over 1M streaming tokens, confirming $O(1)$ inference state.
6. Prefill (full-sequence forward) throughput benchmarking showing that Gravity surpasses Transformer speed at seq=4,096 and reaches $7.5\times$ higher throughput with $99\times$ less memory at seq=16K; at seq=32K, the Transformer throughput-harness memory footprint exceeds A100 40GB while Gravity processes 5.9M tokens/second. This benchmark measures prefill (full-sequence forward) throughput, not token-by-token autoregressive decode.
7. **Identification of the density bottleneck—not the field equation kernel—as the core architectural innovation:** two qualitatively different field solvers (causal Poisson, $O(N \log N)$, versus exponential kernel, $O(N)$) achieve identical test perplexity at seq_len=1,024, indicating that discriminative information resides in the compressed density field rather than the specific long-range propagation mechanism (Section 5.1).

2 Related Work

2.1 Efficient Attention

FlashAttention [4,5] reduces memory by tiled computation but retains $O(N^2)$ FLOPs. Sparse attention methods [1,3] restrict attention to local windows plus selected global tokens, achieving $O(N)$ at the cost of reduced expressiveness. Linear attention [11] approximates softmax with kernel feature maps but often degrades quality. Gravity does not approximate self-attention—it replaces it entirely with field-mediated interaction.

2.2 State-Space Models

Structured state-space models (S4 [9]) and Mamba [8] achieve $O(N)$ via linear recurrence with input-dependent parameters. Mamba-2 [6] establishes a formal duality between SSMs and structured masked attention through semiseparable matrices, showing that scalar-identity SSMs correspond to 1-semiseparable attention. Gravity’s exponential kernel solver is mathematically a scalar SSM—but differs from Mamba in what it propagates and how it uses the result. Mamba propagates 256-dimensional state vectors with input-dependent A, B, C matrices and uses the SSM output directly for prediction. Gravity first compresses K -dimensional parameters to a scalar density $\rho = \sum_k w_k \cdot \alpha_k^2$ via a nonlinear bottleneck, propagates only this scalar through fixed-rate EMA, and then derives a 227-dimensional physics feature vector (density, potential, gradient, entropy, curvature) from the propagated field—using the SSM as a feature generator rather than a

direct predictor. Jamba [12] interleaves Transformer and Mamba layers; Gravity integrates field propagation and local attention within a single block.

2.3 Multi-Dimensional Sequence Models

Extending sequence models to multi-dimensional data is an active area. S4ND [15] generalizes S4 to multi-dimensional signals via outer products of 1D kernels. Vision Transformer (ViT, Dosovitskiy et al. [7]) processes images by flattening 2D patches to a 1D sequence. VMamba [14] introduces SS2D—a 2D selective scan that processes patches along four directions ($\rightarrow$, $\leftarrow$, $\uparrow$, $\downarrow$)—to adapt Mamba for images. Mamba-ND [13] extends Mamba to arbitrary dimensions by alternating row-major orderings across layers: each layer processes one flattened direction (e.g., H_+ one layer, H_- the next, then W_+ , W_-), requiring multiple layers for information to flow in all directions.

Gravity shares the 4-directional scan concept with VMamba and Mamba-ND but differs in three fundamental ways. First, these methods propagate high-dimensional state vectors (Mamba’s $d_{\text{state}} = 16\text{--}256$); Gravity propagates scalar density, a $256\times$ compression. Second, VMamba and Mamba-ND use the SSM output directly; Gravity extracts a composite physics feature vector from the propagated field. Third, Mamba-ND alternates scan directions across layers (each layer sees one direction); Gravity executes all directional scans simultaneously within each layer, so every layer has full multi-directional context. The density bottleneck and physics feature extraction—not the directional scan itself—are Gravity’s core contributions.

2.4 Diffusion Models

Denoising diffusion probabilistic models [10] generate data by iteratively denoising from Gaussian noise. The Diffusion Transformer (DiT [16]) replaces the UNet backbone with a Transformer, achieving state-of-the-art image generation with AdaLN-Zero conditioning. However, DiT inherits Transformer’s $O(N^2)$ attention cost, which becomes prohibitive for high-resolution 2D generation and infeasible for 3D volumetric generation. Gravity-Diffusion replaces DiT’s attention with $O(N)$ field propagation, enabling diffusion at resolutions where quadratic attention cannot operate.

2.5 Physics-Inspired Neural Networks

Physics-informed neural networks (PINNs [17]) use physical laws as soft constraints. Graph neural networks use message-passing on discrete structures. Gravity is distinct: it uses discrete field equations not as regularizers but as the fundamental information propagation mechanism. The field equation’s Green’s function directly determines how density from each element propagates to all other positions.

3 Method

3.1 Overview

The Gravity pipeline processes input data on a D -dimensional lattice through: (1) embedding to d_{model} dimensions; (2) projection to K field parameters via $\text{params} = \pi \cdot \tanh(W \cdot \text{LayerNorm}(x))$, bounded for numerical stability; (3) compression to C -channel scalar density $\rho = \text{softplus}(W_{\text{density}}) \cdot \text{params}^2$; (4) multi-scale exponential field solving via directional parallel scans; (5) local window attention scoring; (6) physics feature extraction; (7) projection back to d_{model} with residual connection; (8) feedforward network with second residual. All steps are $O(N)$ per layer. No $N \times N$ matrix is materialized.

3.2 Density Bottleneck

The density bottleneck is the architectural core. Each element’s K -dimensional field parameters are compressed to C scalar density channels. For single-channel ($C = 1$): $\rho(x) = \sum_k w_k \cdot \alpha_k(x)^2$, where $w_k = \text{softplus}(\text{raw_}w_k)$ ensures non-negative weights. For multi-channel ($C > 1$): $\rho_c(x) = \text{softplus}(W_{\text{density}}) \cdot \text{params}(x)^2$, where W_{density} is a learned $K \times C$ projection. The squared nonlinearity ensures non-negative density (physical constraint: mass is non-negative) and provides a structured inductive bias distinct from linear projections used in SSMs.

This parameterization traces to an $SU(4)$ lattice gauge theory motivation [21] in which each element’s local degrees of freedom are represented by Lie algebra coefficients a_k associated with the 15 generators T_k of $SU(4)$. The scalar density corresponds to a gauge-invariant generator-energy proxy $\rho \propto \text{Tr}(A^\dagger A)/G$ with $A = \sum_k a_k T_k$ (where G is the group dimension, $G = 4$ here), which reduces to a weighted quadratic form in the coefficients—implemented as $\rho = \sum_k w_k a_k^2$. The $\pi \cdot \tanh(\cdot)$ bounding of field parameters corresponds to parameterizing the Lie algebra coefficients within a compact range. The full theoretical derivation, in which linearized Einstein gravity emerges from discrete-time quantum walks with $SU(4)$ coin operators [21], is developed separately and is not required for the empirical results presented here; this paper treats $SU(4)$ as a structural motivation for the $K = 15$ parameterization and density bottleneck.

The compression ratio K/C controls the information–regularization tradeoff. $C = 1$ provides maximum compression ($K:1$ ratio) and strongest regularization; $C = K$ provides no compression. Our experiments identify $C = 16$ with $K = 64$ ($4\times$ compression) as the sweet spot at 100M scale.

3.3 Multi-Scale Exponential Field Solver

For each scale $s = 1, \dots, S$ and channel c , the potential field is computed via first-order recurrence:

$$\phi_{s,c}(t) = \alpha_s \cdot \phi_{s,c}(t-1) + \beta_s \cdot \rho_c(t),$$

where $\alpha_s = \exp(-1/\lambda_s)$ is the learnable decay rate and β_s is the coupling amplitude. This corresponds to a Yukawa-type Green’s function $G(d) = \exp(-d/\lambda)$, providing exponentially decaying influence from past density sources. Multiple scales capture multi-resolution temporal structure: $\lambda \approx 2$ (local, 2–3 tokens), $\lambda \approx 16$ (sentence-level), $\lambda \approx 128$ (paragraph-level). The causal gradient is $\nabla\phi(t) = \phi(t) - \phi(t-1)$.

The recurrence is strictly causal by construction and implemented via parallel associative scan [2]. The binary operator $(\alpha_1, b_1) \otimes (\alpha_2, b_2) = (\alpha_1 \cdot \alpha_2, \alpha_2 \cdot b_1 + b_2)$ enables $O(N)$ work in $O(\log_2 N)$ parallel depth. On A100, the parallel scan achieves $13\times$ speedup over sequential scanning (23 s vs 375 s per epoch at `seq_len=1,024`).

3.4 Multi-Dimensional Extension

The field equation $\nabla^2\phi = \rho$ provides the physical motivation; the implementation uses a Green’s function family of exponential kernel propagators. The dimensional extension is achieved via directional scan decomposition: the D -dimensional field solve decomposes into $2 \cdot D$ one-dimensional parallel scans, with two directions for each axis:

1D (sequences): A single causal scan from position 1 to N . Suitable for language modeling, time series, and audio.

2D (images): Four directional scans: rightward and leftward along each row, downward and upward along each column. For a grid of $H \times W$ positions, each scan is a standard 1D parallel scan applied to H rows (for horizontal) or W columns (for vertical). All four scans execute in parallel. The resulting potential fields are

combined to provide complete spatial context. Total complexity: $O(4 \cdot S \cdot H \cdot W) = O(N)$. Local attention uses a $(2R + 1)^2$ spatial window (e.g., 3×3 for $R = 1$).

3D (volumes): Six directional scans along the positive and negative directions of each axis ($\pm x, \pm y, \pm z$). For a grid of $D \times H \times W$ positions, each scan operates on slices along the respective axis. Total complexity: $O(6 \cdot S \cdot D \cdot H \cdot W) = O(N)$. Local attention uses a $(2R + 1)^3$ window (e.g., $3 \times 3 \times 3 = 27$ neighbors for $R = 1$).

General D dimensions: $2 \cdot D$ directional scans. Total complexity remains $O(N)$ for fixed D . No flattening or artificial scan ordering is required.

Video (2D space + 1D time, 3D spatio-temporal lattice): Five directional scans: four bidirectional spatial scans ($\pm x, \pm y$) and one causal temporal scan (forward only). For a video of T frames on an $H \times W$ spatial grid, each spatial scan processes $T \cdot H$ rows or $T \cdot W$ columns; the temporal scan processes $H \cdot W$ pixel-positions across T frames. Total complexity: $O(5 \cdot S \cdot T \cdot H \cdot W) = O(N)$. The temporal scan is strictly causal—each frame’s field state depends only on current and past frames—enabling online video processing with $O(1)$ temporal state. *Terminology note*: the label “4D video” sometimes used elsewhere in this paper refers to video tensors that include the channel dimension; the field propagation itself operates on the 3D spatio-temporal lattice $T \times H \times W$.

3.5 Local Window Attention

Preliminary diagnostics on trained dense-attention Gravity probes show attention mass highly concentrated near the self-position (approximately 99% on the self-position in the tested WikiText-2 character-level setting). The architecture functions as a physics-informed feature extractor, not a content-based retrieval mechanism. The primary discriminative signal comes from the composite feature vector—density, potential, gradient, attention statistics, and neighbor parameters—rather than attention-weighted value aggregation. This observation directly motivates the local window formulation: since attention mass falls overwhelmingly within the immediate neighborhood in our probes, restricting computation to a fixed window preserves the meaningful signal in this regime while eliminating $O(N^2)$ cost.

For 1D, attention scores are computed within a fixed-size causal window $[t - W + 1, t]$, where $W = 2R + 1$ (default $R = 5$, $W = 11$). The score combines field potential differences, gradient differences, positional distance, and field parameter similarity:

$$\text{scores}(t, i) = -w_\phi |\phi_s(t) - \phi_s(i)| - w_F |\nabla \phi_s(t) - \nabla \phi_s(i)| - w_d \cdot d(t, i) / \varepsilon - w_c \cdot \|\alpha(t) - \alpha(i)\|^2 / \sigma^2.$$

Softmax is applied over W positions. No causal mask is needed: window indices include only past positions by construction. The output is a $[B, S, N, W]$ tensor—never $[B, S, N, N]$.

For 2D and 3D, the same scoring function is applied within a spatial window of $(2R + 1)^D$ neighbors, using potential differences aggregated from all directional fields. The window size is independent of the total lattice size N , maintaining $O(N)$ complexity.

3.6 Physics Feature Extraction

At each position, a composite feature vector is assembled from: (1) $S \times W$ local attention weights; (2) K own field parameters; (3) $W \times K$ neighbor field parameters; (4) S attention entropies; (5) S attention centroids; (6) S tidal curvatures; (7) relative position; (8) local density; (9) running mean density; (10) density z-score; (11) density trend. For multi-dimensional lattices, potential and gradient values from all $2 \cdot D$ directional fields are included. For $C > 1$, $S \times C$ potential values and $S \times C$ gradient values are added. With $K = 15$,

$S = 3$, $W = 11$, this yields 227 features in 1D. All features are computed from $O(N)$ -sized tensors. The feature vector is projected through a 2-layer MLP back to d_{model} .

3.7 Multi-Layer Architecture

The above components are composed into a stackable GravityBlock. Each block performs: LayerNorm $\rightarrow$ field projection $\rightarrow$ density $\rightarrow$ field solve $\rightarrow$ local attention $\rightarrow$ feature extraction $\rightarrow$ feature projection + residual $\rightarrow$ FFN + residual. Multiple blocks are stacked with independent parameters per layer. Depth scaling from 1 to 8 layers shows consistent improvement.

4 Experiments

We organize experiments into three sections: scaling and language modeling (establishing that Gravity works at scale), cross-dimensional universality (demonstrating the unified framework across 1D sequences through spatio-temporal video), and hardware efficiency (proving the practical advantage of $O(N)$ complexity and $O(1)$ state). *Note:* All 100M-scale language experiments (Tables 5–8) are single-epoch data-scaling probes, not fully converged training runs. Small-scale and multi-dimensional experiments are trained to convergence.

Table 1: Gravity compared against attention/Transformer-style or sequence-model baselines across four data regimes. CNN baselines are excluded from this summary comparison because they are task-specialized vision architectures relying on built-in translation invariance; they are reported separately in the 2D/3D sections. Gravity’s goal is a unified cross-dimensional field architecture, not replacing convolutional inductive bias on pure vision tasks. Gravity outperforms the strongest tested attention/Transformer-style baseline on three of five discriminative tasks (sensor, image, volume) and is comparable on the other two (language modeling at $2.1\times$ fewer parameters; video within 0.8 pp), validates $O(N)$ diffusion for generative tasks, and demonstrates $7.5\times$ throughput advantage at 16K context, $92\times$ memory reduction at 32K, Transformer OOM at 65K, and constant-memory streaming over 1M tokens. The “Regime” column denotes the data regime / lattice dimensionality on which field propagation operates; video corresponds to the 3D spatio-temporal lattice $T \times H \times W$. 100M-scale results (Tables 5–8) are 1-epoch data-scaling probes, not fully converged training runs; small-scale and multi-dimensional results are trained to convergence.

Task	Regime	Gravity	Best Attn/SSM Baseline	Result
Language modeling (PPL)	1D	4.36	4.09 (TF 4L)	Competitive at $2.1\times$ fewer params
Sensor RUL (RMSE)	1D	14.37	16.12 (LSTM)	Gravity wins, 11% better
Image classif. (Acc)	2D	82.3%	80.3% (ViT)	Gravity wins, 19% fewer params
Medical volume (Acc)	3D	89.7%	79.0% (ViT-3D)	Gravity wins, 10.7% better
Video classif. (Acc)	Video	88.1%	88.9% (ViT-Video)	Comparable (0.8 pp), 17% fewer params
Image generation (FID)	2D	79.2	63.0 (DiT)	Feasibility validated, $O(N)$ vs $O(N^2)$
Volume generation (32^3)	3D	0.0014	DiT OOM at 32K patches	$O(N)$ validated; Gravity 0.73 GB, DiT OOM
Inference memory (32K)	1D	140 MB	12,902 MB (TF)	$92\times$ less memory
Throughput (16K)	1D	2.5M tok/s	336K tok/s (TF)	$7.5\times$ faster, $99\times$ less memory
Ultra-long seq (65K)	1D	411 MB	OOM (TF)	Transformer cannot run; Gravity can
Streaming 1M tokens	1D	71 ± 0.1 MB	N/A	$O(1)$ state confirmed (CV=0.14%)

4.1 Setup

All experiments use the same core architecture (density bottleneck, exponential kernel field solver, local window attention, physics features) with only the lattice dimensionality and scan directions varying. Hyperparameters are set independently per task but the architectural components are identical.

1D Language Modeling (Small-scale): WikiText-2 character-level (vocab $\approx 1,013$). $d_{\text{model}} = 128$, $K = 15$, $S = 3$, $W = 11$, learning rate 2×10^{-3} with AdamW [24], batch size 8. Trained on Apple M4 Pro (MPS) and NVIDIA A100 40GB (CUDA).

1D Language Modeling (100M scale): OpenWebText [22] with GPT-2 BPE tokenization [23] (vocab=50,257). $d_{\text{model}} = 768$, $K = 64$, $C = 16$, $S = 3$, $W = 11$, 12 layers, learning rate 3×10^{-4} with AdamW [24] (weight decay 0.1), gradient accumulation 4 (effective batch 32), mixed precision (AMP). Trained on NVIDIA A100 40GB.

1D Sensor (C-MAPSS): NASA Turbofan Engine Degradation Simulation [18], FD001 subset. 100 engines run-to-failure, 21 sensor channels. Sequence length 30, $d_{\text{model}} = 64$, $K = 15$, $S = 3$, $R = 3$, 2 layers, learning rate 2×10^{-3} with AdamW [24] (weight decay 1×10^{-4}), 100 epochs. RUL capped at 125 cycles following standard practice.

2D Image (CIFAR-10): 32×32 RGB images, 10 classes. 4×4 patches $\rightarrow 8 \times 8$ grid (64 patches). $d_{\text{model}} = 128$, $K = 15$, $S = 3$, $R = 1$ (3×3 window), 4 layers, learning rate 1×10^{-3} with AdamW [24], 50 epochs. Standard augmentation (random crop, horizontal flip).

3D Medical Volume (OrganMNIST3D): $28 \times 28 \times 28$ CT organ volumes from MedMNIST v2 [20], 11 classes. $4 \times 4 \times 4$ patches $\rightarrow 7 \times 7 \times 7$ grid (343 patches). $d_{\text{model}} = 64$, $K = 8$, $S = 3$, $R = 1$ ($3 \times 3 \times 3$ window), 2 layers, learning rate 2×10^{-3} with AdamW [24], 50 epochs.

Video (Moving MNIST): 10-frame 32×32 grayscale videos of bouncing digits, 10 classes. 4×4 spatial patches $\rightarrow 10 \times 8 \times 8$ spatio-temporal grid (640 tokens). $d_{\text{model}} = 64$, $K = 8$, $S = 3$, 2 layers, learning rate 1×10^{-3} with AdamW [24], 50 epochs. 5-directional scan on the 3D spatio-temporal lattice: 4 spatial bidirectional + 1 temporal causal.

2D/3D Diffusion: DDPM [10] with 1000 timesteps, AdaLN-Zero conditioning [16]. *2D*: CIFAR-10, 8×8 patch grid, 6 Gravity-2D blocks ($d = 256$, $K = 16$), 250 epochs. *3D*: Newcastle Voxel Dataset [19], 50K binary 20^3 volumes, $4 \times 4 \times 4$ patch grid, 6 Gravity-3D blocks ($d = 256$, $K = 16$), 100 epochs.

Part I: Scaling and Language Modeling

4.2 1D Language Modeling: Sequence Length 1,024

Table 2: WikiText-2 character-level, seq_len=1,024, 20 epochs. Gravity 1-layer (555K) is competitive with Transformer 4-layer (1,185K, $2.1 \times$ more parameters) and clearly beats a matched 1-layer Transformer. Transformer baselines use the default PyTorch TransformerEncoder (norm_first=False, GELU activation, 4 heads, dropout 0.1, causal mask) trained with AdamW (lr= 2×10^{-3} , weight decay 0.05, cosine schedule). Gravity numbers in this table are single-seed; Table 21 reports a 3-seed mean (4.53) of the same Gravity configuration used for the field-solver ablation in Section 5.1.

Model	Layers	Params	Test PPL	Time/ep
Gravity	1	555K	4.36	23 s (A100)
Transformer	1	590K	4.95	—
Transformer	2	788K	4.34	—
Transformer	4	1,185K	4.09	—

Table 3: WikiText-2 character-level, seq_len=4,096, 20 epochs, A100. Gravity trains $7\times$ faster with $372\times$ less attention memory while achieving lower perplexity.

Model	Params	Test PPL	Time/ep	Attn Mem
Gravity	948K	4.79	6 s	4.3 MB
Transformer	475K	5.05	50 s	1.61 GB

Table 4: WikiText-2 char-level, seq_len=128, M4 Pro, 40 epochs. Gravity 2-layer outperforms Mamba 1-layer and Transformer 1-layer. Gravity 4-layer matches Transformer 4-layer with 13% fewer parameters. This table uses Apple M4 Pro with a pure-PyTorch Mamba implementation and the shorter sequence length (seq=128); absolute PPL values are not directly comparable to Tables 2–3 (A100, seq=1,024).

Model	Layers	Params	Test PPL	Note
Gravity	1	193K	5.06	
Gravity	2	604K	4.01	beats Mamba
Gravity	4	930K	3.78	beats TF 4L
Mamba	1	522K	4.15	pure PyTorch
Transformer	1	673K	4.21	
Transformer	4	1,070K	3.81	

4.3 1D Language Modeling: Sequence Length 4,096

4.4 Depth Scaling and Mamba Comparison

4.5 100M Scale: Channel Ablation

We scale Gravity to ~ 128 M parameters ($d_{\text{model}} = 768$, $K = 64$, 12 layers) with GPT-2 BPE tokenization [23] on OpenWebText [22]. We ablate the number of density channels C while controlling for parameter count:

Table 5: Channel ablation at 100M scale (OpenWebText BPE, 100M tokens, 1 epoch, A100). $C = 16$ ($4\times$ compression from $K = 64$) is the sweet spot. $C = 32$ shows diminishing returns, consistent with loss of regularization benefit.

Configuration	Params	Test PPL	Time
$K = 15, C = 1$	98M	130.0	35 min
$K = 64, C = 1$	126M	118.3	37 min
$K = 64, C = 16$	128M	103.4	38 min
$K = 64, C = 32$	130M	105.6	46 min
Transformer	124M	86.6	18 min

4.6 100M Scale: Data Scaling and the Compression–Retrieval Tradeoff

To disentangle underfitting from architectural limitations, we increase training data from 100M to 1B tokens for both Gravity ($C = 16$) and Transformer:

Table 6: Data scaling at 100M parameter scale (OpenWebText BPE, 1 epoch, A100). Both models improve substantially with more data. The PPL gap widens from 19% to 41%; however, PPL ratios are misleading at low perplexity values (see Section 4.6.1). Table 7 reports the scale-invariant cross-entropy loss gap, which grows from 3.9% to 9.6% with decelerating rate—the appropriate metric for assessing scaling behavior.

Model	Params	100M tokens	300M tokens	1B tokens
Gravity $C = 16$	128M	103.4	67.0	47.8
Transformer	124M	86.6	48.2	33.9
PPL Gap	—	19.4%	39.0%	41.0%

4.6.1 Cross-Entropy Loss Gap (Scale-Invariant Metric)

Perplexity percentage gaps are amplified at lower PPL values: a model improving from PPL 100 to 50 (50% reduction) achieves only a 0.69 reduction in cross-entropy loss, identical to improving from PPL 50 to 25. This makes PPL ratios misleading for comparing scaling behavior. We therefore report the gap in cross-entropy loss ($= \ln \text{PPL}$), which provides a scale-invariant comparison:

Table 7: Cross-entropy loss gap analysis. The loss gap grows from 3.9% to 9.6%, but the rate of increase decelerates from +4.6 percentage points (100M→300M) to +1.1 percentage points (300M→1B), suggesting convergence toward a stabilizing information cost of the density bottleneck rather than unbounded degradation.

Tokens	Gravity Loss	TF Loss	Loss Gap	Δ from prev
100M	4.64	4.46	3.9%	—
300M	4.20	3.87	8.5%	+4.6 pp
1B	3.87	3.52	9.6%	+1.1 pp

This deceleration is the central empirical finding regarding Gravity’s scaling behavior. The density bottleneck imposes an information cost that is currently consistent with stabilization near $\sim 10\%$ in cross-entropy loss space rather than a compounding penalty. Transformer retains per-token key/value representations for all N tokens, enabling content-addressable retrieval; Gravity compresses context to $S \cdot C$ scalar field states, losing the ability to recall specific token content. At small scale with character-level tokenization, the density bottleneck’s regularization outweighs its information loss. At 100M scale with BPE, information loss dominates: each BPE token carries more semantic content, making precise retrieval more valuable. The multi-channel ablation (Table 5) shows this tradeoff has a measurable optimum: $C = 16$ ($4\times$ compression) maximizes the information–regularization balance.

4.6.2 Sequence Length Scaling

To investigate the effect of sequence length on the quality gap, we train both models at three sequence lengths with matched optimizer steps ($\sim 3,000$ steps, effective batch size 32), scaling the token count proportionally so each configuration sees approximately one epoch of data:

Part II: Cross-Dimensional Universality

4.7 1D Sensor: NASA C-MAPSS RUL Prediction

To validate Gravity beyond language modeling, we apply it to remaining useful life (RUL) prediction of turbofan engines using NASA’s C-MAPSS FD001 dataset [18]. This is a naturally limited-data scenario: 100

Table 8: Sequence length scaling at 100M parameter scale (OpenWebText BPE, $\sim 3,000$ optimizer steps, A100). The absolute cross-entropy gap narrows from 0.168 to 0.035 nats as sequence length increases from 1,024 to 10,240, corresponding to a relative CE gap reduction from 3.8% to 0.9%. Both models are undertrained at $\sim 3,000$ steps (compared to $\sim 30,000$ steps in the data scaling experiments of Tables 6–7); we report the trend here as suggestive evidence that longer sequences narrow the gap, with confirmation at convergence left to future work.

Seq Length	Tokens	Gravity PPL	TF PPL	Δ CE (nats)	Relative CE
1,024	100M	102.2	86.4	0.168	3.8%
4,096	400M	69.6	59.9	0.150	3.7%
10,240	1B	58.7	56.7	0.035	0.9%

training engines with 21 sensor channels, predicting cycles until failure.

Table 9: NASA C-MAPSS FD001 RUL prediction ($d_{\text{model}} = 64$, 2 layers, 100 epochs, A100). Gravity achieves the best RMSE (11% better than LSTM), best MAE, and best NASA S-Score (39% better than LSTM). The S-Score penalizes late predictions (predicting failure after it occurs) more heavily than early predictions; Gravity’s lower S-Score indicates fewer dangerous late predictions. The density bottleneck provides natural regularization on this small dataset.

Model	Params	RMSE	MAE	S-Score
Gravity	94K	14.37	10.83	327
LSTM	57K	16.12	12.24	537
Transformer	105K	16.39	12.59	613

4.8 2D Image Classification: CIFAR-10

We extend Gravity to two-dimensional data by replacing the single causal scan with 4-directional parallel scans ($\rightarrow \nwarrow \nearrow \uparrow$) on the 2D patch grid. The density bottleneck, local window attention, and physics feature extraction are identical to the 1D version—only the field solver changes.

Table 10: CIFAR-10 image classification ($d_{\text{model}} = 128$, 4 layers, 50 epochs, A100). Gravity-2D outperforms ViT-Tiny by 2.0% with 19% fewer parameters. CNN achieves the highest accuracy due to built-in translation invariance but exhibits significantly more overfitting (train-test gap 9.9% vs Gravity’s 2.5%). The density bottleneck provides natural regularization against overfitting.

Model	Type	Params	Test Acc	Train-Test Gap
CNN	Conv2d	559K	89.7%	9.9%
Gravity-2D	4-dir	653K	82.3%	2.5%
ViT-Tiny	Attention	809K	80.3%	3.8%

4.9 3D Medical Volume Classification: OrganMNIST3D

We further extend Gravity to three-dimensional data using 6-directional parallel scans ($\pm x, \pm y, \pm z$) on a 3D patch grid. The dataset is OrganMNIST3D from MedMNIST v2 [20]: $28 \times 28 \times 28$ CT organ volumes with 11 organ classes.

Table 11: OrganMNIST3D medical volume classification ($d_{\text{model}} = 64$, 2 layers, 50 epochs, A100). Gravity-3D outperforms ViT-3D by 10.7% with 10% fewer parameters. The margin over ViT is larger in 3D (+10.7%) than in 2D (+2.0%), confirming that multi-directional field propagation provides increasingly stronger spatial inductive bias relative to vanilla attention in higher dimensions.

Model	Type	Params	Test Acc	Note
CNN-3D	Conv3d	418K	96.1%	built-in 3D bias
Gravity-3D	6-dir	114K	89.7%	+10.7% vs ViT
ViT-3D	Attention	127K	79.0%	flattened to 1D

4.10 Video Classification: Moving MNIST

To validate Gravity in the spatio-temporal domain, we extend the architecture to the 3D spatio-temporal lattice with 5-directional field propagation: 4 bidirectional spatial scans (right, left, down, up) and 1 causal temporal scan (forward only). The temporal scan ensures that each frame’s field state depends only on current and past frames. Although classification allows bidirectional temporal access, we deliberately impose temporal causality to demonstrate suitability for real-time online video processing—where future frames are unavailable by definition. The dataset is Moving MNIST: 10-frame 32×32 videos of bouncing digits (10 classes, 10K train, 2K test).

Table 12: Moving MNIST video classification ($d = 64$, 2 layers, 50 epochs, A100). Gravity-Video with 3×3 spatial local window attention achieves 88.1% accuracy, comparable to ViT-Video (88.9%) within noise (0.8 pp) using 17% fewer parameters. The 5-directional scan provides spatio-temporal context without flattening the lattice to 1D. CNN-3D achieves the highest accuracy due to built-in translation invariance, consistent with the 2D and 3D classification results.

Model	Params	Acc	Complexity
CNN-3D	279K	90.0%	$O(N)$ conv
ViT-Video	143K	88.9%	$O(N^2)$ attn
Gravity-Video	118K	88.1%	$O(N)$ scan

4.11 2D Image Generation (Diffusion)

To validate Gravity beyond discriminative tasks, we apply Gravity-2D as the denoiser backbone in a DDPM diffusion model [10] for unconditional image generation on CIFAR-10 (32×32 RGB). The architecture replaces DiT’s $O(N^2)$ self-attention with Gravity’s $O(N)$ 4-directional field propagation. Timestep conditioning uses AdaLN-Zero [16], identical to DiT. Images are patchified to an 8×8 grid (patch size 4), processed through 6 Gravity-2D blocks ($d = 256$, $K = 16$), and unpatchified to predict noise.

4.12 3D Volume Generation (Diffusion)

We further extend Gravity-Diffusion to three-dimensional generation using 6-directional field propagation. Two experiments validate Gravity’s 3D generative capability: (1) shape quality on synthetic 32^3 binary volumes, and (2) memory scaling comparison against DiT-3D at increasing patch counts on 64^3 volumes.

Shape Generation (32^3). The dataset consists of 10,000 synthetic binary 32^3 volumes from 5 geometric classes (sphere, cube, torus, cylinder, ellipsoid). Volumes are patchified to an $8 \times 8 \times 8$ grid (patch size 4),

Table 13: CIFAR-10 unconditional generation (DDPM 1000 steps, 250 epochs, A100, no EMA). Apples-to-apples comparison: identical training setup (optimizer, schedule, epochs, batch size), only the denoiser backbone differs. Gravity-Diffusion achieves FID 79.2 versus DiT’s 63.0. Neither model uses EMA in this baseline comparison; the asymmetric impact of EMA on these two architectures is discussed in Section 6.

Model	Params	FID	Complexity
Gravity-Diffusion	6.3M	79.2	$O(N)$ per step
DiT Baseline	7.8M	63.0	$O(N^2)$ per step

processed through 6 Gravity-3D blocks ($d = 192$, $K = 16$, $S = 3$) with AdaLN-Zero timestep conditioning, pred-x0 clipping, and EMA (decay=0.999).

Table 14: 3D shape generation quality (32^3 , ps=4, 200 epochs, A100, 3.8M params, loss $0.178 \rightarrow 0.0014$). Generated volumes match ground-truth occupancy, connectivity, and surface statistics. An auxiliary experiment at 64^3 with ps=8 failed to converge (pure noise output), suggesting that patch embedding capacity is a limiting factor at this resolution: when each patch must represent $8^3 = 512$ voxels in 192 dimensions ($2.7\times$ compression), the information loss exceeds what Gravity recovers in this configuration. Whether larger embedding dimensions, multi-scale tokenization, or VAE-based latent representations close the gap is left to future work; the architecture itself functions correctly whenever the patch embedding preserves sufficient information (e.g., ps=4 yields $4^3 = 64$ voxels $\rightarrow$ 192 dim, a $3\times$ expansion).

Metric	Generated	Training GT	64^3 ps=8 (failed)
Occupancy	0.05 ± 0.00	0.04 ± 0.00	0.50 (noise)
Connected components	1.25 ± 0.4	1.00 ± 0.0	2568 (fragments)
Surface area	948 ± 409	918 ± 400	387K (speckle)

Memory Scaling (64^3 , DiT-3D vs Gravity-3D). To demonstrate Gravity’s $O(N)$ memory advantage for 3D generation, we compare peak GPU memory on 64^3 volumes at decreasing patch sizes. Both models use matched architecture ($d = 192$, 6 layers), inference only (batch=2, float32).

A note on the ViT/Transformer comparisons in this section: ViT is known to underperform at small scale due to its lack of spatial inductive bias [7], requiring large-scale pretraining to achieve its full potential. Gravity’s advantage over ViT at small scale therefore reflects the density bottleneck’s role as a physics-derived spatial inductive bias—analogueous to how convolution provides translation equivariance. The comparison demonstrates not that Gravity is universally superior to ViT, but that field-equation propagation provides structured inductive bias that pure attention lacks, enabling competitive performance without large-scale pretraining.

Part III: Hardware Efficiency

4.13 Memory Scaling

4.14 Training Speed

4.15 Inference Memory Benchmark

To empirically validate the theoretical $O(N)$ memory scaling, we measure GPU memory consumption during inference at context lengths from 512 to 32,768 tokens for Gravity ($d = 128$, 4 layers) and Transformer ($d = 128$, 4 layers, 4 heads) on A100.

Table 15: 3D diffusion memory scaling on A100 40GB (64^3 volume, batch=2, float32). At ps=2, DiT-3D’s $O(N^2)$ attention requires a $32,768 \times 32,768$ matrix (1.07 billion entries, ~ 4.3 GB in fp32 as a lower bound, before accounting for batch size, attention heads, layers, Q/K/V activations, softmax buffers, and framework overhead). In the measured 6-layer, batch=2 setting, DiT-3D exceeds A100 40GB memory, while Gravity-3D’s $O(N)$ field propagation uses only 0.73 GB at the same resolution. The memory ratio grows from $1.1\times$ at 512 patches to infinity at 32K patches—the regime where 3D generation at fine resolution is only possible with $O(N)$ architectures.

ps	Patches	DiT-3D	Gravity	Ratio
8	512	0.16 GB	0.15 GB	$1.1\times$
4	4,096	1.25 GB	0.21 GB	$5.9\times$
2	32,768	OOM	0.73 GB	∞

Table 16: Attention tensor memory scaling (batch_size=8, $S = 3$ scales, float32). Gravity’s linear scaling enables deployment at context lengths where quadratic methods exceed available memory.

seq_len	Dense $O(N^2)$	Gravity $O(N)$	Ratio
1,024	100.7 MB	1.1 MB	$93\times$
4,096	1.61 GB	4.3 MB	$372\times$
8,192	6.44 GB	8.7 MB	$745\times$
128K	1.6 TB	137 MB	$11,924\times$

The theoretical $O(1)$ autoregressive inference state per layer is $S + W \times K = 3 + 11 \times 15 = 168$ scalars = 672 bytes. At 4 layers: 2,688 bytes = 2.6 KB, independent of context length. This enables deployment on devices where Transformer’s linearly growing KV-cache is infeasible.

4.15.1 Prefill / Full-Sequence Forward Throughput Benchmark

To complement the memory analysis, we measure prefill (full-sequence forward) throughput in tokens per second for both architectures at increasing sequence lengths on A100, using 1-layer models ($d = 128$) with batch size 1. This benchmark measures the cost of processing an entire input sequence in a single forward pass—analogous to a Transformer’s prefill phase—and is distinct from token-by-token autoregressive decode throughput. Streaming decode memory is evaluated separately in Section 4.18.

Full-Scale Prefill Throughput (12 layers, $d = 768$). To validate that the $O(N)$ advantage holds at production scale, we repeat the prefill throughput benchmark using the full 125M-parameter architecture (12 layers, $d = 768$, $K = 64$, $C = 16$) on A100 40GB with batch size 1.

4.16 Window Size Ablation

To determine the minimum local window size, we sweep R from 0 to 16 (window $W = 2R + 1$) on WikiText-2 character-level (seq=1024, 1 layer, $d = 128$, $K = 15$, lr= 3×10^{-4} , 20 epochs):

4.17 Ultra-Long Sequence Stress Test

To directly demonstrate the practical consequence of $O(N)$ vs $O(N^2)$ scaling, we push sequence length from 1,024 to 65,536 tokens on a single A100 40GB GPU, measuring whether each model can complete a forward pass.

Table 17: Training time per epoch (small-scale char-level). Parallel scan: $13\times$ speedup over sequential scanning on A100. At seq_len=4,096, Gravity is $8\times$ faster than Transformer. The crossover point is approximately seq_len=2,048. *Note:* epoch time is measured over a fixed corpus, so increasing seq_len reduces the number of batches per epoch; consequently wall-clock epoch time is not a per-token latency metric. Per-token throughput is reported separately in Table 19.

Implementation	A100 seq=1,024	A100 seq=4,096	M4 Pro seq=1,024
Gravity (sequential)	375 s	N/A	89 s
Gravity (parallel scan)	23 s	6 s	23 s
Transformer	7 s	50 s	48 s

Table 18: Inference memory scaling (batch=1, $d = 128$, 4 layers, A100). Gravity exhibits approximately linear scaling after fixed runtime overheads; Transformer scales quadratically. At 32K context, Gravity uses $92\times$ less memory. At very short contexts, absolute GPU memory is dominated by fixed CUDA/PyTorch allocator overhead and kernel workspace, so the seq=512 point is not monotonic with the seq=1,024 point; the asymptotic linear trend is visible from 1K onward. The crossover with Transformer occurs around 1K tokens. Extrapolation beyond measured sequence lengths depends on implementation details such as FlashAttention, activation checkpointing, and memory allocator behavior; the measured results up to 32K already demonstrate the empirical crossover and a $92\times$ memory ratio. Baseline is vanilla dense PyTorch attention; FlashAttention reduces attention memory by tiled computation but retains $O(N^2)$ FLOPs.

Context Length	Gravity	Transformer	Ratio
512	12.7 MB	4.6 MB	$0.4\times$
1,024	5.3 MB	13.1 MB	$2.5\times$
4,096	18.4 MB	203.5 MB	$11.1\times$
16,384	70.6 MB	3,230 MB	$45.8\times$
32,768	140.2 MB	12,902 MB	$92.0\times$

4.18 Streaming $O(1)$ State Verification

To empirically verify the $O(1)$ inference state claim, we stream 1,000,000 characters through a trained Gravity model (1 layer, $d = 128$, seq=1,024) in sliding windows of 1,024 tokens with stride 512. Every 50 windows, we measure GPU memory, per-window latency, and perplexity.

4.19 Inference State Comparison

The most distinctive property of Gravity is its $O(1)$ autoregressive inference state. During generation, each layer maintains only S scalar field states and a $W \times K$ circular buffer of recent field parameters. With default settings ($S = 3$, $W = 11$, $K = 15$), this is 168 scalars: **672 bytes in fp32, or 336 bytes in fp16**. By contrast, a Transformer with $d_{\text{model}} = 4096$ at 128K context stores $128K \times 4096 \times 2 \approx 1.07$ billion values per layer (≈ 4.3 GB in fp32, 2.15 GB in fp16). At 32 layers, Transformer requires ≈ 137 GB for KV-cache alone in fp32; Gravity requires ≈ 21 KB in fp32. This $\sim 6 \times 10^6\times$ difference is the basis of Gravity’s deployment value.

To contextualize this tradeoff: conventional KV-cache quantization achieves $4\text{--}16\times$ compression with 2–10% quality loss, while Gravity achieves $\sim 10^6\times$ compression at 128K context with an observed cross-entropy cost near $\sim 10\%$ in our current scaling range. The compression ratio grows linearly with sequence length while the observed quality cost stays in the same approximate range—a qualitatively different operating

Table 19: Prefill throughput and memory scaling (1 layer, $d = 128$, batch=1, A100). The speed crossover occurs at seq=4,096: below this, Transformer is faster due to optimized attention kernels; above, Gravity’s $O(N)$ scaling dominates. At seq=16K, Gravity is $7.5\times$ faster and uses $99\times$ less memory. At seq=32K, the Transformer throughput harness exceeds A100 40GB memory while Gravity processes 5.9M tokens/second using 327 MB. *Note on harness:* this throughput benchmark retains additional intermediate tensors (timing buffers, autograd-mode allocations) compared to the pure-forward-pass benchmark of Table 22, which is why Transformer at seq=32K OOMs here but completes in Table 22 with 12,930 MB. Table 18 (4-layer forward, no throughput instrumentation) and Table 22 (1-layer pure forward) should be treated as the canonical inference-memory benchmarks; Table 19 reports throughput under a measurement harness whose absolute memory values are not directly comparable. Gravity’s throughput increases with sequence length (parallel scan efficiency improves with longer sequences), while Transformer’s throughput decreases (quadratic attention cost dominates).

Seq Length	Gravity tok/s	Gravity MB	TF tok/s	TF MB	Speed	Memory
512	144,678	70 MB	714,637	78 MB	$0.20\times$	$1.1\times$
1,024	273,685	74 MB	1,365,939	112 MB	$0.20\times$	$1.5\times$
2,048	539,603	83 MB	1,405,594	375 MB	$0.38\times$	$4.5\times$
4,096	1,003,438	99 MB	831,230	1,287 MB	$1.21\times$	$13.0\times$
8,192	1,843,828	132 MB	497,096	4,923 MB	$3.71\times$	$37.3\times$
16,384	2,530,638	196 MB	336,078	19,444 MB	$7.53\times$	$99.1\times$
32,768	5,912,879	327 MB	OOM	OOM	—	—

regime from fixed-ratio quantization techniques. We are not aware of lossy compression methods that operate at comparable compression ratios with single-digit percentage quality loss.

5 Analysis

5.1 Density Bottleneck Equivalence

Two qualitatively different field solvers—causal Poisson ($G(d) = d$, $O(N \log N)$) and exponential kernel ($G(d) = \exp(-d/\lambda)$, $O(N)$)—achieve identical test PPL of 4.53 at seq_len=1,024 (3-seed mean). The linear Green’s function weights distant tokens more heavily; the exponential decays locally. Their equivalence implies that discriminative information resides in the density field ρ and its local structure, not in the specific long-range propagation kernel. This identifies the density bottleneck—not the field equation—as the core innovation.

This finding has two consequences. First, it justifies the choice of the cheaper $O(N)$ exponential kernel for all subsequent experiments and for deployment: there is no measurable quality cost relative to the $O(N \log N)$ Poisson solver. Second, it reframes the relationship between Gravity and prior linear-recurrence methods: what distinguishes Gravity from a scalar SSM is not the recurrence dynamics, but the upstream density bottleneck that compresses K -dimensional field parameters to a scalar before propagation. The “physics-inspired” framing earns its empirical weight here—not from any specific field equation being uniquely correct, but from the density-bottleneck-plus-propagation pattern providing a structured inductive bias that linear projections in SSMs do not.

Table 20: Full-scale prefill throughput (12 layers, $d = 768$, 125M params, batch=1, FP32, A100 40GB). Same measurement harness as Table 19. The speed crossover occurs earlier at seq=2,048 (compared to seq=4,096 for the 1-layer model), because the deeper architecture amplifies the $O(N^2)$ vs $O(N)$ cost difference across 12 layers. At seq=4,096, Gravity is $2.1\times$ faster; at seq=8,192, $4.0\times$ faster. Beyond seq=8,192, the Transformer throughput harness exceeds A100 memory while Gravity continues to seq=32,768 at 64K tokens/second using only 7.5 GB.

Seq Length	Gravity tok/s	Gravity MB	TF tok/s	TF MB	Speed	Memory
128	3,182	571 MB	24,233	558 MB	$0.13\times$	$1.0\times$
512	12,002	653 MB	45,657	640 MB	$0.26\times$	$1.0\times$
1,024	22,193	762 MB	44,777	748 MB	$0.50\times$	$1.0\times$
2,048	38,824	978 MB	34,461	1,449 MB	$1.13\times$	$1.5\times$
4,096	50,101	1,413 MB	23,563	4,109 MB	$2.13\times$	$2.9\times$
8,192	56,746	2,279 MB	14,046	14,667 MB	$4.04\times$	$6.4\times$
16,384	61,332	4,021 MB	OOM	OOM	—	—
32,768	64,253	7,495 MB	OOM	OOM	—	—

Table 21: Window size ablation (WikiText-2, seq=1024, 1L). $W = 7$ ($R = 3$) achieves the same PPL as the default $W = 11$ ($R = 5$), indicating the field propagation provides sufficient long-range context that only a small local window is needed. At $R = 0$ (field only, no local window), PPL degrades to 11.64, confirming that local attention remains essential for fine-grained token interactions.

R	W	Params	Test PPL
0	1	532K	11.64
1	3	537K	5.64
2	5	541K	4.64
3	7	546K	4.53
5	11	555K	4.53
8	17	569K	4.53
16	33	606K	4.55

5.2 Regularization Advantage

Across all tasks, Gravity exhibits smaller train-test gaps than ViT and Transformer, consistent with the density bottleneck providing natural regularization:

5.3 Causal Integrity

Leakage testing confirms zero future information leakage. The exponential kernel solver is causal by construction: $\phi(t)$ depends only on $\phi(t-1)$ and $\rho(t)$. The local window attention uses only past indices $[t-W+1, t]$. Empirical verification: modifying future tokens produces zero change in earlier logits (KL divergence $< 2 \times 10^{-6}$).

6 Discussion and Limitations

Several limitations should be noted.

1. A quality gap exists at scale: at 1B training tokens, the cross-entropy loss gap between Gravity and

Table 22: Ultra-long sequence stress test (1 layer, $d = 128$, batch=1, forward pass, A100 40GB). Note: this experiment uses a different model configuration (1 layer) than Table 18 (4 layers); absolute memory values are not directly comparable. The purpose of this table is feasibility: at seq=65,536, Transformer exceeds GPU memory and cannot execute, while Gravity completes the forward pass using 411 MB—less than 1% of GPU capacity.

seq_len	Gravity	Transformer	Ratio
1,024	18 MB	25 MB	$1.4\times$
2,048	24 MB	64 MB	$2.7\times$
4,096	36 MB	217 MB	$6.0\times$
8,192	62 MB	825 MB	$13.3\times$
16,384	112 MB	3,250 MB	$29.0\times$
32,768	211 MB	12,930 MB	$61.3\times$
65,536	411 MB	OOM	∞

Table 23: Streaming inference over 1M tokens (A100). Memory coefficient of variation is 0.14%—effectively constant, confirming $O(1)$ state. Latency variation (19%) is attributable to GPU scheduling jitter. By contrast, a Transformer processing 1M tokens would require a KV-cache of $1\text{M} \times 128 \times 2 \times 4$ bytes = 1.0 GB per layer, growing linearly with each new token. Gravity’s state remains at 672 bytes per layer regardless of how many tokens have been processed.

Metric	Mean	Std	CV
GPU Memory	71 MB	0.1 MB	0.14%
Latency per Window	4.2 ms	0.8 ms	19%
Perplexity	stable	stable	stable

Transformer is 9.6% (PPL 47.8 vs 33.9). This gap decelerates from +4.6 pp to +1.1 pp across the measured data-scale increases (100M $\rightarrow$ 300M $\rightarrow$ 1B tokens), suggesting convergence toward an information cost consistent with stabilization rather than unbounded degradation, but it remains a meaningful quality difference at current scale.

2. CNNs outperform Gravity-2D/3D on image and volume tasks due to built-in translation invariance and weight sharing. CNNs can be adapted to sequences (1D CNN, TCN, WaveNet, ConvS2S), but standard convolutional architectures are task-specialized and do not provide the same unified mechanism for causal long-context language modeling, spatial grids, volumetric data, and streaming video. Gravity’s value is a unified cross-domain field framework, not beating CNNs on pure vision tasks.
3. Training speed is $2\times$ slower than Transformer at seq_len=1,024 due to parallel scan overhead; speed advantage only emerges at seq_len $\geq$ 4,096.
4. All large-scale experiments use 1 epoch; multi-epoch training may shift the tradeoff.
5. Mamba comparison is limited to seq_len=128 with pure PyTorch implementation.
6. The 2D and 3D classification experiments use small-scale datasets; scaling behavior for multi-dimensional discriminative tasks is untested. The diffusion experiments validate $O(N)$ feasibility but do not target state-of-the-art FID. Weight-averaging techniques such as EMA substantially improve DiT (FID

Table 24: Inference state comparison (fp32 unless noted). Gravity’s state is approximately $200\times$ smaller than Mamba ($398\times$ smaller in fp16) and approximately $6 \times 10^6\times$ smaller than a 32-layer Transformer at 128K context.

	Gravity	Mamba	Transformer
State per layer (fp32)	672 bytes	~ 130 KB	$2 \times N \times d_{\text{model}}$ values (GB scale)
128K context, 32 layers (fp32)	~ 21 KB	~ 4.2 MB	~ 137 GB
Ratio vs Transformer (fp32)	$\sim 6 \times 10^6\times$ smaller	$\sim 33,000\times$ smaller	$1\times$
Ratio vs Gravity (fp32)	$1\times$	$\sim 200\times$ larger	$\sim 6 \times 10^6\times$ larger

Table 25: Regularization comparison. Gravity consistently generalizes better than ViT/Transformer. This advantage is particularly valuable in limited-data scenarios common in industrial and medical applications.

Task	Gravity gap	ViT/TF gap
CIFAR-10	2.5%	3.8% (ViT)
OrganMNIST3D	8.0%	20.6% (ViT-3D)
C-MAPSS (val $\rightarrow$ test)	1.5 RMSE	3.3 RMSE (TF)

63 $\rightarrow$ 51) but have negligible effect on Gravity (FID 79 $\rightarrow$ 79), confirming that the generation quality gap stems from the density bottleneck’s information compression rather than training instability. Addressing this gap likely requires mechanisms that recover high-frequency information lost during density compression, such as the wormhole retrieval described in Future Work.

7. The 3D shape generation quality is demonstrated only at 32^3 resolution; higher-resolution 3D generation (64^3+) requires patch sizes that preserve sufficient information per patch, a constraint shared by all patch-based architectures.
8. The scalar density bottleneck discards the matrix structure of the underlying $SU(G)$ representation (where G is the gauge/group dimension, distinct from sequence length N)—phase information, non-commutative structure, and pairwise coupling among generator coefficients are lost in reducing the quadratic form $\text{Tr}(A^\dagger A)/G$ to a weighted sum of squared coefficients. This may account for a portion of the $\sim 10\%$ cross-entropy gap that cannot be closed by engineering improvements alone.

Despite these limitations, Gravity establishes results of independent value. The density bottleneck provides sufficient inductive bias for competitive performance across four data regimes, demonstrating that compressing representations to scalar density before field propagation is a viable and general design principle. The $O(1)$ autoregressive inference state provides a concrete advantage for memory-constrained deployment. While other architectures such as Mamba-ND also process multi-dimensional data with $O(N)$ complexity, Gravity’s density bottleneck achieves a qualitatively different operating point: approximately $6 \times 10^6\times$ smaller inference state than a 32-layer Transformer (fp32, 128K context) and approximately $200\times$ smaller than Mamba per layer, at the cost of $\sim 10\%$ cross-entropy loss. This extreme compression–efficiency tradeoff is Gravity’s distinctive contribution.

Implicit content selectivity (hypothesis). We hypothesize that the density mechanism may act as an implicit importance weighting during training. Each token’s scalar density value modulates the strength of its contribution to the surrounding field: tokens generating higher density would produce stronger field potentials that propagate further and exert greater influence on neighboring predictions. During backpropagation, tokens with higher prediction loss would generate larger gradients that flow back through the field solver to the

density computation, potentially reinforcing the model’s tendency to assign higher density to positions where prediction is difficult. If this dynamic holds, the architecture would naturally concentrate its representational capacity on informationally challenging positions without external supervision—in contrast to Transformer attention, which distributes computation uniformly across all token pairs regardless of information content. A practical implication of this hypothesis, if confirmed, is that external data optimization strategies (such as curriculum learning or token importance weighting) may provide greater benefit to architectures that lack built-in content selectivity than to architectures with such selectivity. Quantitative validation of this hypothesis—via density-loss correlation analysis and density visualization—is left to future work.

Future directions. We identify three categories of future work, ordered by expected impact.

Closing the quality gap.

0. **Matrix-valued field propagation:** replacing scalar density with full $SU(G)$ matrix field states, where G denotes the gauge/group dimension (independent of sequence length N). The recurrence becomes $\Phi(t) = U(t) \cdot \Phi(t-1) \cdot U(t)^\dagger + \beta \cdot C(t)$ with U derived from input parameters. For example, $SU(4)$ requires a fixed 4×4 matrix state per scale per layer, preserving $O(1)$ autoregressive state. This recovers the group structure lost in scalar compression. Matrix operations map naturally to GPU tensor cores, potentially improving both quality (more information in the field state) and hardware utilization (compute-bound rather than memory-bound).
1. **Density-gated sparse retrieval (“wormhole” attention):** a fixed-size memory bank where high-density tokens are stored for content-addressable retrieval, providing $O(N \cdot M)$ long-range access without $O(N^2)$ cost.
2. **Hybrid architectures** interleaving Transformer and Gravity layers, where Gravity layers handle long-range context propagation and Transformer layers provide precise content retrieval where needed.

Scaling and deployment.

3. Scaling to 1B+ parameters with multi-epoch training to characterize the loss gap at production scale.
4. Custom CUDA/Triton kernels for the parallel scan.
5. Dedicated hardware acceleration via analog RC circuits and neuromorphic processors.
6. Streaming deployment benchmark: continuous sensor or video monitoring with $O(1)$ state over days or weeks.
7. Gravity-specific training strategies including progressive sequence length training and cross-dimensional transfer learning.

New domains.

8. High-resolution image and video processing at 4K/8K where attention-based methods exceed memory limits.
9. Higher-dimensional structured data: 5D+ climate modeling, multi-channel neurophysiological signals, computational fluid dynamics.
10. Extension to irregular lattices (graphs) for social network analysis and point cloud processing.

7 Conclusion

We presented Gravity, a neural network framework that replaces $O(N^2)$ self-attention with $O(N)$ physics-derived field equations operating natively on lattices of any dimensionality. The density bottleneck—compressing K -dimensional field parameters to C -channel scalar density via nonlinear projection—provides the core inductive bias, enabling $O(N)$ computation, $O(N)$ memory, and $O(1)$ autoregressive inference state per layer.

Across eleven experiments in four data regimes, Gravity outperforms the strongest tested attention/Transformer-style baseline on three of five discriminative tasks (sensor, image, volume), is comparable on language modeling (PPL 4.36 vs 4.09 with $2.1\times$ fewer parameters) and video (within 0.8 pp), and validates $O(N)$ generative feasibility where DiT exceeds GPU memory. The cross-dimensional extension requires no change to the core propagation mechanism—only the number of directional scans varies—yet the 2D, 3D, and video instances all achieve competitive discriminative accuracy at matched or lower parameter count. At 100M parameter scale, the cross-entropy loss gap between Gravity and Transformer grows from 3.9% to 9.6% with sharply decelerating rate, consistent with convergence toward a stabilizing information cost near $\sim 10\%$ rather than unbounded degradation.

The efficiency results are decisive: $92\times$ memory reduction at 32K context, $7.5\times$ throughput advantage at 16K, Transformer OOM at 65K while Gravity uses 411 MB, and constant-memory streaming over 1M tokens (CV=0.14%). The $O(1)$ inference state—168 scalars per layer versus millions of KV-cache entries—provides a path to memory-constrained deployment at arbitrarily long context lengths.

The density bottleneck and physics feature extraction—not the directional scan—are what distinguish Gravity from other multi-dimensional SSMs. While VMamba and Mamba-ND also achieve $O(N)$ complexity across dimensions, Gravity’s scalar compression yields $O(1)$ inference state (672 bytes/layer vs Mamba’s ~ 130 KB/layer) at the cost of $\sim 10\%$ cross-entropy loss: a qualitatively different point in the compression–retrieval tradeoff that, to our knowledge, is not matched by prior architectures.

Furthermore, the equivalence of two qualitatively different field solvers (causal Poisson, $O(N \log N)$, and exponential kernel, $O(N)$) at identical test perplexity (Section 5.1) identifies the density bottleneck—not the specific field equation—as the source of Gravity’s empirical strength. This reframes the contribution: Gravity is not a paper about one particular physics-derived recurrence; it is a paper about scalar density compression as a structured inductive bias that field propagation then operationalizes.

Gravity is not intended to replace task-specialized inductive biases such as CNNs on pure vision classification or DiT-style models for highest-fidelity image generation. Its value is a different operating point: $O(N)$ field propagation, $O(1)$ streaming state along causal dimensions, and one density-field mechanism spanning sequences, grids, volumes, and spatio-temporal lattices.

Code, Data, and Contact

Code and data availability. Reference implementations of Gravity-1D, Gravity-2D, Gravity-3D, and the parallel associative scan, together with reproduction scripts and trained checkpoints for the small-scale benchmarks reported in Tables 2, 10, and 11, are available at <https://github.com/chiangjw90/gravity-nn>. All datasets used in this paper are publicly available: WikiText-2, OpenWebText, CIFAR-10, MedMNIST v2 (OrganMNIST3D), NASA C-MAPSS FD001, Moving MNIST, and the Newcastle Voxel Dataset.

Correspondence. Correspondence regarding this work, including potential collaboration or commercial licensing inquiries, may be directed to the author at: chiangjw90@gmail.com.

References

- [1] Beltagy, I., Peters, M. E., & Cohan, A. (2020). Longformer: The long-document transformer. *arXiv:2004.05150*.
- [2] Bllloch, G. E. (1990). Prefix sums and their applications. Technical Report CMU-CS-90-190, Carnegie Mellon University.
- [3] Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating long sequences with sparse transformers. *arXiv:1904.10509*.
- [4] Dao, T. (2023). FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv:2307.08691*.
- [5] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS 2022*.
- [6] Dao, T., & Gu, A. (2024). Transformers are SSMS: Generalized models and efficient algorithms through structured state space duality. *arXiv:2405.21060*.
- [7] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An image is worth 16×16 words: Transformers for image recognition at scale. *ICLR 2021*.
- [8] Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*.
- [9] Gu, A., Goel, K., & Ré, C. (2022). Efficiently modeling long sequences with structured state spaces. *ICLR 2022*.
- [10] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *NeurIPS 2020*.
- [11] Katharopoulos, A., Vyas, A., Pappas, N., & Flückiger, F. (2020). Transformers are RNNs: Fast autoregressive transformers with linear attention. *ICML 2020*.
- [12] Lieber, O., Lenz, B., Bata, H., et al. (2024). Jamba: A hybrid Transformer-Mamba language model. *arXiv:2403.19887*.
- [13] Li, S., Singh, H., & Grover, A. (2024). Mamba-ND: Selective state space modeling for multi-dimensional data. *ECCV 2024*.
- [14] Liu, Y., Tian, Y., Zhao, Y., et al. (2024). VMamba: Visual state space model. *arXiv:2401.10166*.
- [15] Nguyen, E., Goel, K., Gu, A., Downs, G., Shah, P., Dao, T., Baccus, S., & Ré, C. (2022). S4ND: Modeling images and videos as multidimensional signals with state spaces. *NeurIPS 2022*.
- [16] Peebles, W., & Xie, S. (2023). Scalable diffusion models with transformers. *ICCV 2023*.
- [17] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686–707.

- [18] Saxena, A., & Goebel, K. (2008). Turbofan engine degradation simulation data set. *NASA Prognostics Data Repository*, NASA Ames Research Center.
- [19] Towers, D., Ericsson, L., Crowley, E. J., Atapour-Abarghouei, A., & McGough, A. S. (2024). Newcastle Voxel Dataset. *Figshare*.
- [20] Yang, J., Shi, R., Wei, D., et al. (2023). MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10(41).
- [21] Chiang, C.-W. (2026). Emergent Einstein Gravity from Discrete-Time Quantum Walks with $SU(4)$ Coin Operators. *Zenodo*, v1.0.0, April 19, 2026. <https://doi.org/10.5281/zenodo.19653137>
- [22] Gokaslan, A., & Cohen, V. (2019). OpenWebText Corpus. <https://skylion007.github.io/OpenWebTextCorpus/>
- [23] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Technical Report*.
- [24] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *ICLR 2019*.